ML-Based Cyber Threat Prediction

Project Outline

Domain: Cybersecurity and Machine Learning

By

NAGA GIREESH REDDY (24STUCHH010432)

MANGALI VISHNU DATTA (24STUCHH010201)

DWARAMPUDI NIHAR REDDY (24STUCHH010217)

THALUPURI MANIDEEP SATHYA (24STUCHH010104)

Under the Guidance of

Mr. M. Rama Krishnam Raju

FACULTY OF SCIENCE & TECHNOLOGY

The ICFAI Foundation for Higher Education, Hyderabad



Project Outline

1. Project Title

ML-Based Cyber Threat Prediction — An integrated Machine Learning system for network intrusion and threat detection, addressing zero-day attack detection, encrypted traffic analysis, alert prioritisation, and adversarial robustness.

2. Objective(s)

The project aims to design and implement an ML-based network threat detection system that improves on the specific gaps identified in current research, rather than replicating a single existing approach. Specifically, the project aims to:

- Detect previously unseen (zero-day) attack patterns using a hybrid anomaly-detection and classification architecture, rather than relying solely on known attack signatures.
- Analyze encrypted (TLS/QUIC) network traffic for threats using flow and handshake metadata, without decrypting payload content.
- Reduce alert fatigue by ranking detected threats by confidence and severity, rather than presenting every prediction as an equal-priority alert.
- Improve resilience against adversarial evasion attempts through constrained adversarial retraining and a classifier architecture chosen for its empirical resistance to evasion.
- Present all of the above through an explainable (SHAP-based), analyst-usable monitoring dashboard rather than a black-box prediction engine.

3. Scope

This project is scoped to a working prototype demonstrating the four capabilities above on public network-traffic datasets, not a production-grade enterprise deployment. The following are explicitly included and excluded:

Included:

- A hybrid Autoencoder + XGBoost/Random Forest detection core, trained and evaluated on public intrusion-detection datasets (e.g., NSL-KDD, CICIDS2017/2018).
- Feature engineering for encrypted traffic (packet-size sequences, timing, TLS/JA3-JA4 handshake attributes) so the same detection core functions on encrypted flows without decryption.
- A SHAP-based explainability layer and a risk-ranked alert dashboard (React.js front end, FastAPI backend, Elasticsearch for event storage).
- A constrained adversarial-retraining pipeline to harden the classifier against evasion attempts, evaluated on generated adversarial samples.

Excluded (documented as future work, not attempted in this project):

- Full enterprise SIEM/SOAR tool integration and multi-console data ingestion.
- Graph Neural Network-based flow correlation across an entire live network topology.
- Federated Learning across multiple organizations.
- Endpoint Detection and Response (EDR) / host-agent development (Network Detection and Response scope only).
- Defense against AI-generated, self-rewriting malware (an acknowledged open industry problem, not solved by any current published system).

4. Project Timeline

The timeline below is organized into six modules over an indicative 13-week period. Exact dates should be adjusted to align with the official submission deadline set by the department; Phase 1 (literature review) has already been completed as of this document.

Phase / Module	Duration	Key Tasks
Phase 1 — Literature Review & Proposal	Weeks 1-2	Identify the four problem areas, review current literature, finalize proposed architecture and tech stack (completed).
Phase 2 — Data Collection & Preprocessing	Weeks 3-4	Acquire and clean NSL-KDD/CICIDS datasets; perform feature engineering for both standard flow features and TLS/handshake metadata.
Phase 3 — Core Model Development	Weeks 5-7	Build and tune the Autoencoder anomaly layer and XGBoost/Random Forest classifier; validate on held-out and zero-shot test splits.
Phase 4 — Explainability & Dashboard	Weeks 8-9	Integrate SHAP explainability; build the FastAPI backend, Elasticsearch storage, and React.js risk-ranked alert dashboard.
Phase 5 — Adversarial Hardening	Weeks 10-11	Generate constrained adversarial samples; retrain and re-evaluate the classifier; document before/after robustness results.
Phase 6 — Testing, Documentation & Submission	Weeks 12-13	End-to-end system testing, performance evaluation, final report writing, and preparation for submission ahead of the project deadline.

5. Resources and Tools

Software:

- Programming Language: Python (data processing, model development, backend integration).
- Machine Learning: Scikit-learn, XGBoost, SHAP (explainability).
- Network Data Handling: Scapy / PyShark (packet capture and feature extraction).
- Backend: FastAPI (REST API layer connecting frontend, database, and ML model).
- Frontend: React.js (interactive risk-ranked monitoring dashboard).
- Database: Elasticsearch (storage and fast querying of high-volume traffic logs and predictions).
- Data Processing & Visualization: Pandas, NumPy, Matplotlib, Plotly.
- Development Environment: Jupyter Notebook, Visual Studio Code.
- Version Control: Git and GitHub.

Hardware:

- Standard development laptops/desktops for model training and development (a GPU is not strictly required, since the core models are tree-ensemble and Autoencoder based rather than large deep-learning architectures).
- Sufficient local or cloud storage for public network-traffic datasets (several GB depending on dataset selection).

Datasets:

- NSL-KDD and CICIDS2017/2018 for network intrusion detection and zero-day evaluation.
- Publicly available TLS/encrypted traffic session data (e.g., via Suricata-style logs) for encrypted-traffic feature extraction.

6. Methodology / Approach

The project's technical contribution is the integration of four separately validated research directions into a single working system, rather than a single novel algorithm. The detection core is a two-tier hybrid: an Autoencoder learns a baseline of normal network behavior and flags high-reconstruction-error traffic as anomalous, and this flagged traffic is passed to an XGBoost/Random Forest classifier that determines whether it matches a known attack category or should be treated as a novel threat. This tree-ensemble core is deliberately retained (rather than replaced with a deeper neural network) because current adversarial-evasion research shows tree-based classifiers are empirically harder to evade than deep models.

To function on encrypted traffic without decrypting it, the same detection core is fed engineered metadata features — packet-size sequences and timing (SPLT), protocol/flag combinations, and TLS handshake attributes (cipher suite, JA3/JA4 fingerprint) — rather than payload content, following the metadata-driven approach validated in current Encrypted Traffic Analysis research.

To make the system's output usable by a human analyst rather than adding to alert fatigue, each prediction's confidence score and anomaly score are combined with alert severity and asset criticality into a single composite priority score, which drives a tiered triage flow on the dashboard: high-confidence predictions are auto-logged or immediately flagged as critical depending on severity, while medium-confidence or borderline predictions are placed in an analyst review queue, each accompanied by a SHAP-generated explanation of which specific features drove the prediction.

Finally, to improve resilience against deliberate evasion, a set of adversarial samples is generated against the trained classifier using constrained perturbations that respect real network feature-dependency rules (so generated samples remain valid, executable traffic), and these are mixed into the training data for a retraining pass, following the same before/after evaluation methodology used in current adversarial-robustness research.

7. Conclusion

This project aims to deliver a working ML-based network threat detection prototype that goes beyond a single-purpose classifier by directly addressing four specific, current weaknesses in existing systems: failure to detect zero-day attacks, loss of visibility on encrypted traffic, operationally unusable alert volumes, and vulnerability to adversarial evasion. Success will be measured by the detection core's accuracy on held-out and zero-shot (unseen-attack) test data, the system's ability to classify encrypted traffic sessions using only metadata, a measurable reduction in effective alert volume through the risk-ranking dashboard, and a measurable improvement in adversarial-sample recognition after retraining.

The project deliberately does not claim to solve every open problem in this space. Enterprise-scale tool integration, network-wide graph correlation, federated learning across organizations, endpoint-level detection, and defense against AI-generated malware are acknowledged as valuable extensions beyond this project's current scope, and are documented as future work rather than attempted within the current timeline.